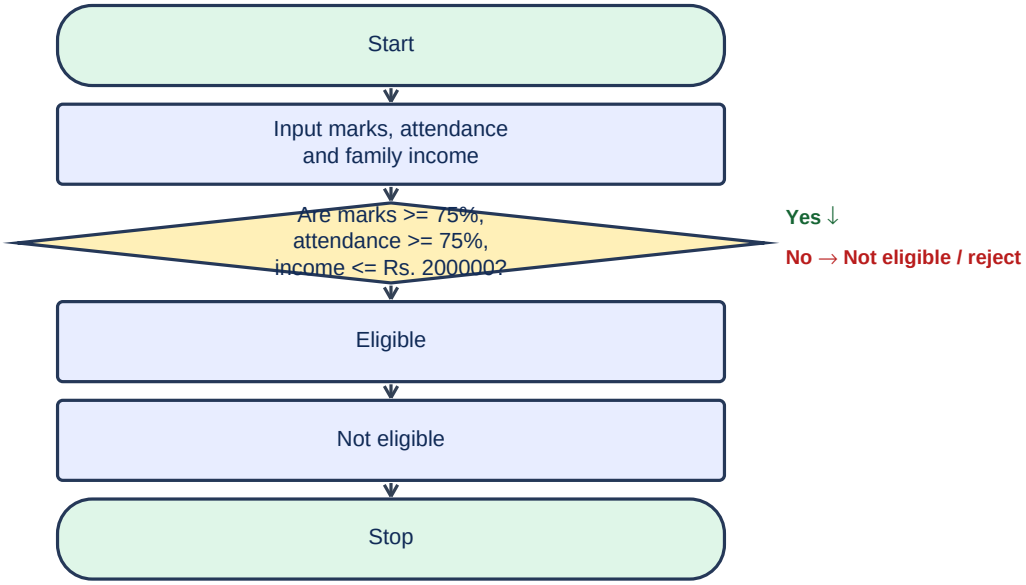
# 1. University Scholarship Allocation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Check if a student qualifies for a scholarship using marks, attendance and family income.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
marks = float(input("Marks: "))
att = float(input("Attendance: "))
income = float(input("Family income: "))
if marks >= 75 and att >= 75 and income <= 200000:
print("Eligible")
else:
print("Not eligible")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Marks: 85, Attendance: 90, Family income: 150000
Output: Eligible

## 5. CONCLUSION

Checks all three scholarship conditions.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Marks: 85
Attendance: 90
Family income: 150000
Eligible
>>
```

# 1. University Scholarship Allocation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To apply multiple conditions together using the logical operator and.
- To make a decision with if-else based on marks, attendance and family income.
- To read numeric input and display an eligibility result.

## 8. ALGORITHM

1. START.
2. Read marks, attendance and family income.
3. IF marks >= 75 AND attendance >= 75 AND income <= 200000 THEN display "Eligible".
4. ELSE display "Not eligible".
5. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT marks, attendance, income
  IF marks >= 75 AND attendance >= 75 AND income <= 200000
    PRINT "Eligible"
  ELSE
    PRINT "Not eligible"
  ENDIF
END
```

## 10. ASSUMPTIONS

- The three values are entered as numbers (marks and attendance as percentages out of 100).
- The scholarship rule is: marks >= 75, attendance >= 75 and family income <= Rs. 200000.
- All three conditions must be true; failing any one makes the student not eligible.
- The program does not check for negative or out-of-range input.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Normal | 85, 90, 150000 | Eligible | Eligible | Pass |
| T2 | Marks below 75 | 70, 90, 150000 | Not eligible | Not eligible | Pass |
| T3 | Attendance below 75 | 85, 70, 150000 | Not eligible | Not eligible | Pass |
| T4 | Income above limit | 85, 90, 250000 | Not eligible | Not eligible | Pass |
| T5 | Boundary | 75, 75, 200000 | Eligible | Eligible | Pass |

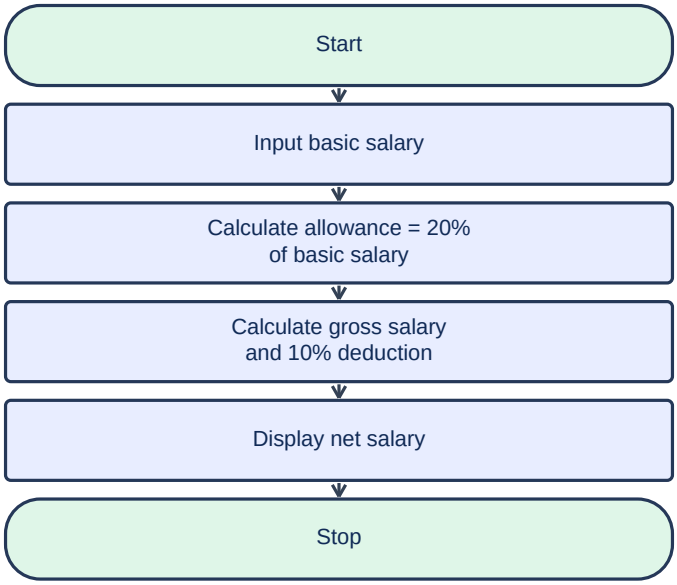
# 2. Employee Payroll Analysis

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Calculate allowance, gross salary, deduction and net salary from the basic salary.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
basic = float(input("Basic salary: "))
allowance = basic * 0.20
gross = basic + allowance
deduction = gross * 0.10
print("Net salary:", gross - deduction)
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Basic salary: 20000
Output: Net salary: 21600.0

## 5. CONCLUSION

Uses arithmetic operations to calculate net salary.

# 2. Employee Payroll Analysis

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To perform arithmetic operations on a salary value.
- To calculate allowance, gross salary, deduction and net salary step by step.
- To understand how one calculated value is used in the next calculation.

## 8. ALGORITHM

1. START.
2. Read the basic salary.
3. Calculate allowance = basic x 0.20.
4. Calculate gross salary = basic + allowance.
5. Calculate deduction = gross x 0.10.
6. Calculate net salary = gross - deduction and display it.
7. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT basic
  allowance <- basic * 0.20
  gross <- basic + allowance
  deduction <- gross * 0.10
  net <- gross - deduction
  PRINT net
END
```

## 10. ASSUMPTIONS

- The basic salary is entered as a positive number in rupees.
- Allowance is a fixed 20% of the basic salary.
- The 10% deduction is applied on the gross salary (basic + allowance), not on the basic salary.
- No other taxes, bonuses or deductions are considered.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Normal | 20000 | Net salary: 21600.0 | Net salary: 21600.0 | Pass |
| T2 | Normal | 10000 | Net salary: 10800.0 | Net salary: 10800.0 | Pass |
| T3 | Large value | 50000 | Net salary: 54000.0 | Net salary: 54000.0 | Pass |
| T4 | Zero salary | 0 | Net salary: 0.0 | Net salary: 0.0 | Pass |

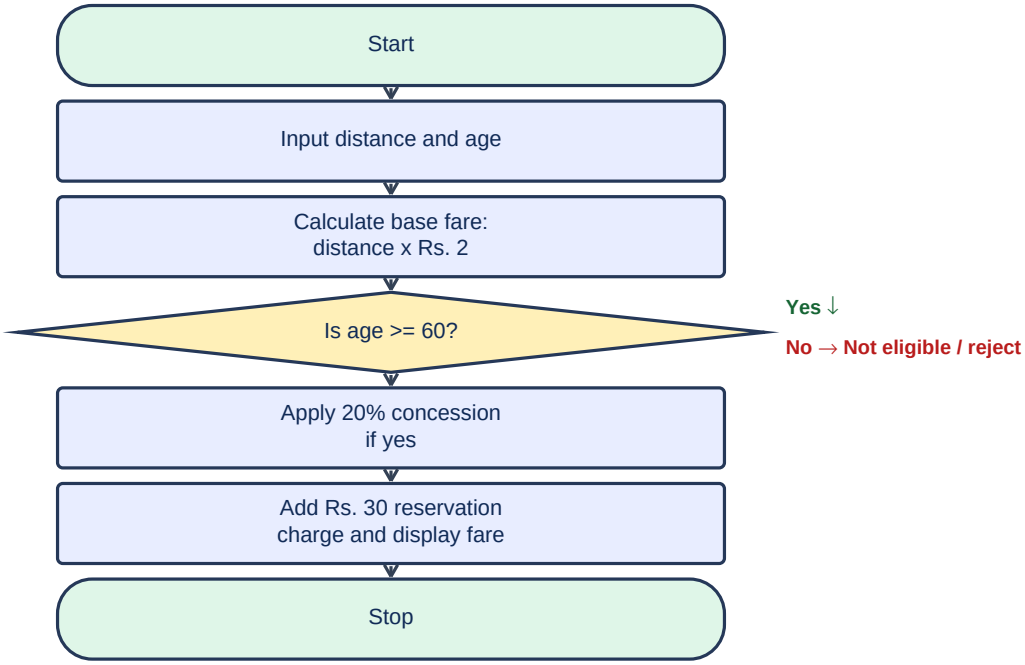
# 3. Railway Fare Calculator

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Calculate an illustrative railway fare using distance and a sample senior-passenger concession.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
km = float(input("Distance in km: "))
age = int(input("Age: "))
fare = km * 2
if age >= 60:
fare = fare * 0.8
print("Final fare:", fare + 30)
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Distance: 100 km, Age: 65
Output: Final fare: 190.0

## 5. CONCLUSION

Demonstrates a sample fare calculation with a condition.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Distance in km: 100
Age: 65
Final fare: 190.0
>>>
```

# 3. Railway Fare Calculator

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To use a condition (age >= 60) to change a calculated value.
- To calculate a fare using multiplication and a percentage concession.
- To combine input, calculation and a fixed extra charge in one program.

## 8. ALGORITHM

1. START.
2. Read the distance in km and the age of the passenger.
3. Calculate fare = distance x 2.
4. IF age >= 60 THEN fare = fare x 0.8 (20% concession).
5. Add Rs. 30 reservation charge to the fare.
6. Display the final fare.
7. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT km, age
  fare <- km * 2
  IF age >= 60 THEN
    fare <- fare * 0.8
  ENDIF
  fare <- fare + 30
  PRINT fare
END
```

## 10. ASSUMPTIONS

- The fare rate is Rs. 2 per km for every passenger (illustrative value).
- A 20% concession applies when the age is 60 or above.
- A fixed reservation charge of Rs. 30 is added after the concession.
- Distance is entered as a positive number and age as a whole number.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Senior citizen | 100, 65 | Final fare: 190.0 | Final fare: 190.0 | Pass |
| T2 | Below 60 | 100, 30 | Final fare: 230.0 | Final fare: 230.0 | Pass |
| T3 | Boundary age 60 | 50, 60 | Final fare: 110.0 | Final fare: 110.0 | Pass |
| T4 | Zero distance | 0, 25 | Final fare: 30.0 | Final fare: 30.0 | Pass |

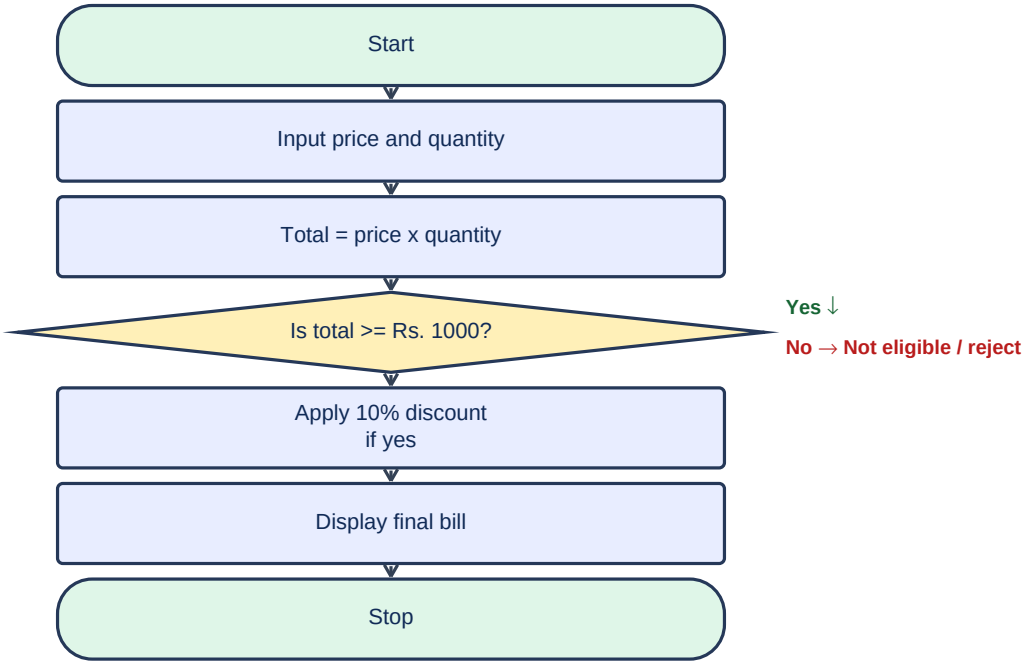
# 4. Retail Billing with Discount

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Calculate the shopping total and apply a discount if the total reaches Rs. 1000.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
price = float(input("Price: "))
qty = int(input("Quantity: "))
total = price * qty
if total >= 1000:
total = total * 0.90
print("Final bill:", total)
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Price: 500, Quantity: 3
Output: Final bill: 1350.0

## 5. CONCLUSION

Calculates a bill and applies a 10% discount when eligible.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Price: 500
Quantity: 3
Final bill: 1350.0
>>>
```

# 4. Retail Billing with Discount

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To calculate a total from price and quantity.
- To apply a discount only when a condition is satisfied.
- To display the final bill after the decision.

## 8. ALGORITHM

1. START.
2. Read the price and the quantity.
3. Calculate total = price x quantity.
4. IF total >= 1000 THEN total = total x 0.90 (10% discount).
5. Display the final bill.
6. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT price, qty
  total <- price * qty
  IF total >= 1000 THEN
    total <- total * 0.90
  ENDIF
  PRINT total
END
```

## 10. ASSUMPTIONS

- Price is the cost of one item and quantity is a whole number of items.
- A 10% discount applies only when the total is Rs. 1000 or more.
- No tax or delivery charge is included in the bill.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Discount applies | 500, 3 | Final bill: 1350.0 | Final bill: 1350.0 | Pass |
| T2 | No discount | 200, 2 | Final bill: 400.0 | Final bill: 400.0 | Pass |
| T3 | Boundary 1000 | 1000, 1 | Final bill: 900.0 | Final bill: 900.0 | Pass |
| T4 | Zero quantity | 100, 0 | Final bill: 0.0 | Final bill: 0.0 | Pass |

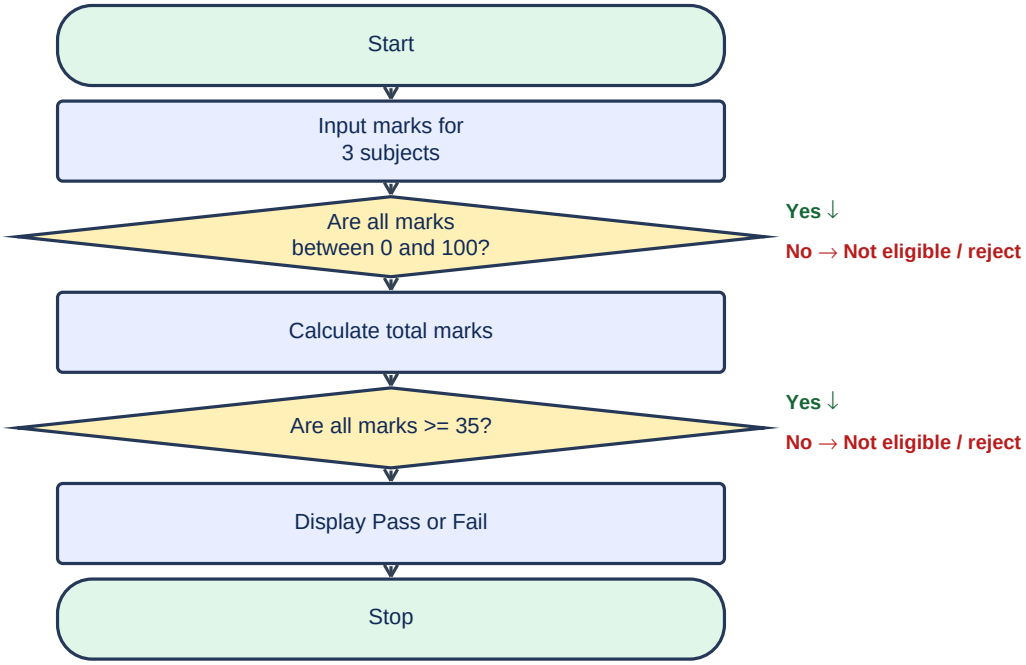
# 5. Examination Result Validation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Calculate total marks for three subjects and check pass or fail.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
a = int(input("Subject 1 marks: "))
b = int(input("Subject 2 marks: "))
c = int(input("Subject 3 marks: "))
total = a + b + c
print("Total:", total)
print("Pass" if min(a,b,c) >= 35 else "Fail")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Marks: 80, 70, 90
Output: Total: 240; Pass

## 5. CONCLUSION

Adds marks and checks the minimum passing mark in each subject.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Subject 1 marks: 80
Subject 2 marks: 70
Subject 3 marks: 90
Total: 240
Pass
>>
```

# 5. Examination Result Validation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To read the marks of several subjects and calculate the total.
- To check the minimum pass mark in every subject using min().
- To display the Pass or Fail result with a conditional expression.

## 8. ALGORITHM

1. START.
2. Read the marks of subject 1, subject 2 and subject 3.
3. Calculate total = subject 1 + subject 2 + subject 3 and display it.
4. IF the lowest of the three marks >= 35 THEN display "Pass".
5. ELSE display "Fail".
6. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT a, b, c
  total <- a + b + c
  PRINT total
  IF minimum(a, b, c) >= 35 THEN
    PRINT "Pass"
  ELSE
    PRINT "Fail"
  ENDIF
END
```

## 10. ASSUMPTIONS

- Each subject is out of 100 and marks are whole numbers between 0 and 100 (the code does not validate this).
- The pass mark is 35 in each subject.
- A student passes only if every subject mark is at least 35, regardless of the total.
- Exactly three subjects are considered.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | All pass | 80, 70, 90 | Total: 240; Pass | Total: 240; Pass | Pass |
| T2 | Boundary 35 | 35, 35, 35 | Total: 105; Pass | Total: 105; Pass | Pass |
| T3 | One subject fails | 80, 34, 90 | Total: 204; Fail | Total: 204; Fail | Pass |
| T4 | Full marks | 100, 100, 100 | Total: 300; Pass | Total: 300; Pass | Pass |
| T5 | All zero | 0, 0, 0 | Total: 0; Fail | Total: 0; Fail | Pass |

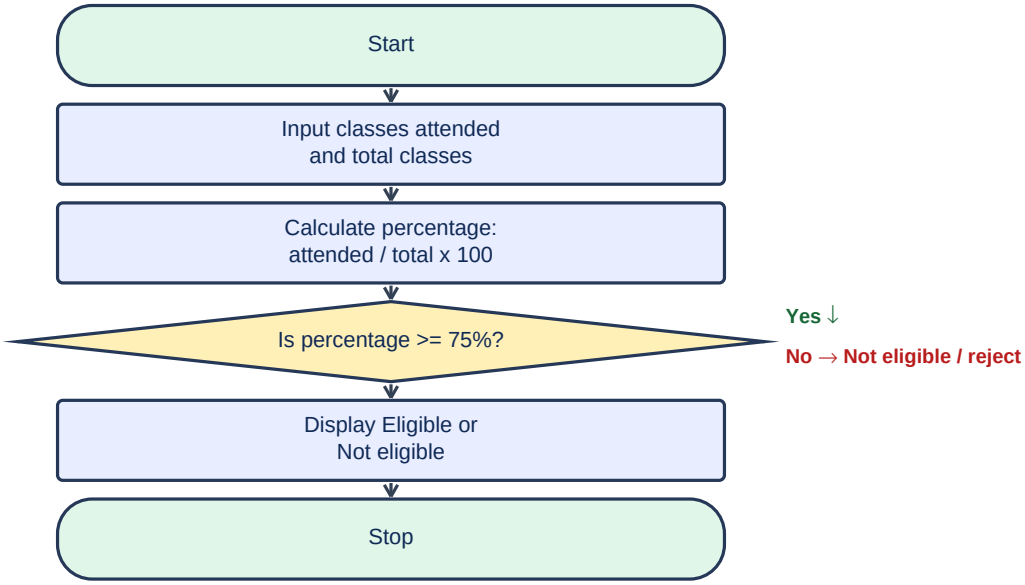
# 6. Attendance Eligibility

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Calculate attendance percentage and check examination eligibility.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
attended = int(input("Classes attended: "))
total = int(input("Total classes: "))
percentage = attended / total * 100
print("Attendance:", percentage, "%")
print("Eligible" if percentage >= 75 else "Not eligible")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Classes attended: 80, Total classes: 100

Output: Attendance: 80.0%; Eligible

## 5. CONCLUSION

Calculates attendance percentage and compares it with 75%.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Classes attended: 80
Total classes: 95
Attendance: 84.21052631578947 %
Eligible
>>>
```

# 6. Attendance Eligibility

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To calculate a percentage from two integer inputs.
- To compare the percentage with the minimum requirement of 75%.
- To decide examination eligibility using a conditional expression.

## 8. ALGORITHM

1. START.
2. Read the classes attended and the total classes.
3. Calculate percentage = (attended / total) x 100 and display it.
4. IF percentage >= 75 THEN display "Eligible".
5. ELSE display "Not eligible".
6. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT attended, total
  percentage <- attended / total * 100
  PRINT percentage
  IF percentage >= 75 THEN
    PRINT "Eligible"
  ELSE
    PRINT "Not eligible"
  ENDIF
END
```

## 10. ASSUMPTIONS

- Classes attended and total classes are whole numbers, and attended <= total.
- Total classes is greater than zero (otherwise division by zero occurs).
- The minimum attendance for examination eligibility is 75%.
- The percentage is displayed without rounding.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Eligible | 80, 100 | Attendance: 80.0 %; Eligible | Attendance: 80.0 %; Eligible | Pass |
| T2 | Boundary 75% | 75, 100 | Attendance: 75.0 %; Eligible | Attendance: 75.0 %; Eligible | Pass |
| T3 | Not eligible | 60, 100 | Attendance: 60.0 %; Not eligible | Attendance: 60.0 %; Not eligible | Pass |
| T4 | Fractional | 80, 95 | Attendance: 84.21052631578947 %; Eligible | Attendance: 84.21052631578947 %; Eligible | Pass |

Note: total classes = 0 is not handled by the program (it raises ZeroDivisionError), so it is not listed as a test case.

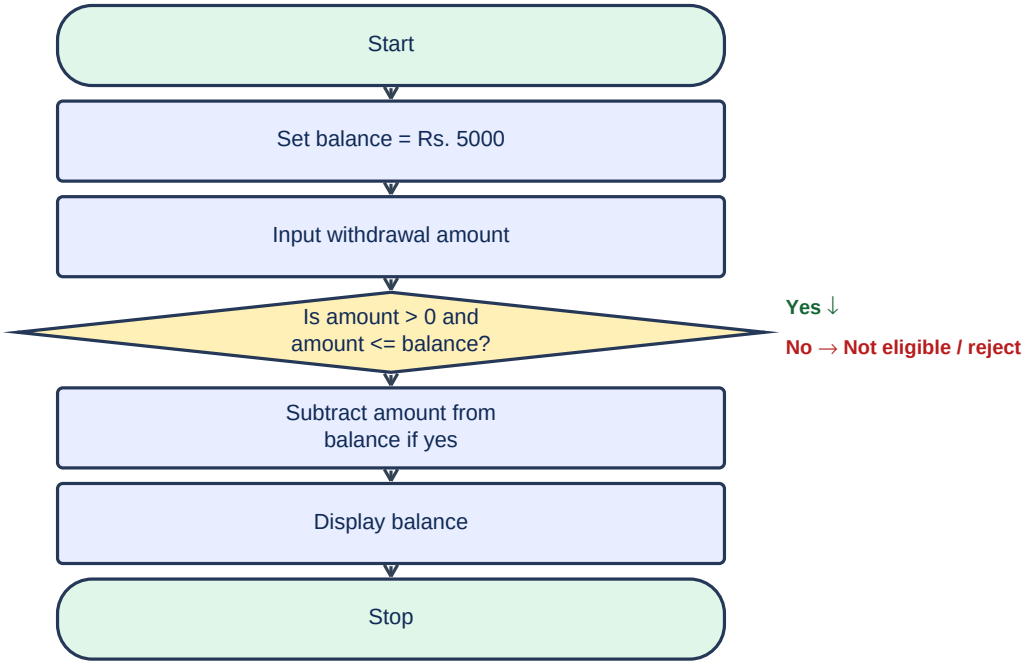
# 7. ATM Withdrawal Simulation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Simulate an ATM withdrawal and prevent withdrawals greater than the balance.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
balance = 5000
amount = float(input("Withdraw amount: "))
if 0 < amount <= balance:
balance = balance - amount
print("Balance:", balance)
else:
print("Invalid amount or insufficient balance")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Withdraw amount: 1200
Output: Balance: 3800.0

## 5. CONCLUSION

Checks the withdrawal amount and updates the balance.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Withdraw amount: 1500
Balance: 3500.0
>>>
```

# 7. ATM Withdrawal Simulation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To simulate an ATM withdrawal with a starting balance.
- To validate the amount before updating the balance.
- To use a chained comparison (0 < amount <= balance) with if-else.

## 8. ALGORITHM

1. START.
2. Set balance = 5000.
3. Read the withdrawal amount.
4. IF amount > 0 AND amount <= balance THEN balance = balance - amount and display the balance.
5. ELSE display "Invalid amount or insufficient balance".
6. STOP.

## 9. PSEUDOCODE

```
BEGIN
  balance <- 5000
  INPUT amount
  IF amount > 0 AND amount <= balance THEN
    balance <- balance - amount
    PRINT balance
  ELSE
    PRINT "Invalid amount or insufficient balance"
  ENDIF
END
```

## 10. ASSUMPTIONS

- The opening account balance is fixed at Rs. 5000.
- The withdrawal amount is a number entered by the user; only one withdrawal is processed per run.
- A withdrawal is allowed only when it is greater than zero and not more than the balance.
- No daily limit, minimum balance or transaction fee is considered.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Valid withdrawal | 1200 | Balance: 3800.0 | Balance: 3800.0 | Pass |
| T2 | Withdraw full balance | 5000 | Balance: 0.0 | Balance: 0.0 | Pass |
| T3 | More than balance | 5001 | Invalid amount or insufficient balance | Invalid amount or insufficient balance | Pass |
| T4 | Zero amount | 0 | Invalid amount or insufficient balance | Invalid amount or insufficient balance | Pass |
| T5 | Negative amount | -100 | Invalid amount or insufficient balance | Invalid amount or insufficient balance | Pass |

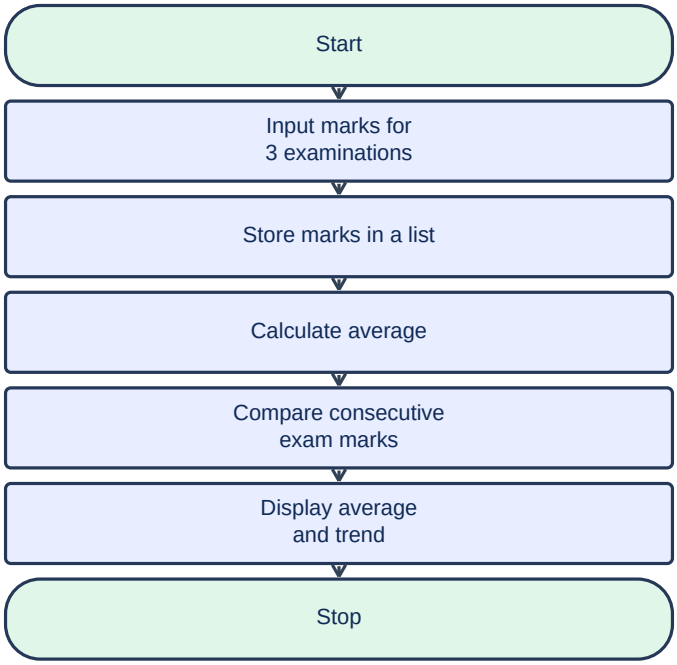
# 8. Student Performance Trend

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Compare marks from three examinations and calculate the average.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
marks = [int(input("Exam mark: ")) for i in range(3)]
print("Average:", sum(marks) / 3)
for i in range(1, 3):
if marks[i] > marks[i-1]: print("Improved")
elif marks[i] < marks[i-1]: print("Declined")
else: print("No change")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Marks: 60, 75, 85

Output: Average: 73.33; Improved; Improved

## 5. CONCLUSION

Uses a list and loop to compare examination performance.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Exam mark: 100
Exam mark: 89
Exam mark: 99
Average: 96.0
Declined
Improved
>>
```

# 8. Student Performance Trend

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To store several values in a list.
- To use a for loop with indexes to compare consecutive values.
- To calculate the average and identify the trend (improved, declined or no change).

## 8. ALGORITHM

1. START.
2. Read the marks of three examinations and store them in a list.
3. Calculate average = sum of marks / 3 and display it.
4. FOR each exam from the second to the third, compare its mark with the previous exam's mark.
5. IF the mark is higher display "Improved"; IF lower display "Declined"; ELSE display "No change".
6. STOP.

## 9. PSEUDOCODE

```
BEGIN
  FOR i <- 1 TO 3
    INPUT marks[i]
  ENDFOR
  PRINT sum(marks) / 3
  FOR i <- 2 TO 3
    IF marks[i] > marks[i-1] THEN PRINT "Improved"
    ELSE IF marks[i] < marks[i-1] THEN PRINT "Declined"
    ELSE PRINT "No change"
  ENDFOR
END
```

## 10. ASSUMPTIONS

- Exactly three examinations are compared, in the order they were entered.
- Marks are whole numbers; each exam is compared only with the one before it.
- Average is the simple mean of the three marks.
- The first examination has nothing before it, so no trend is shown for it.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Improving | 60, 75, 85 | Average: 73.33333333333333; Improved; Improved | Average: 73.33333333333333; Improved; Improved | Pass |
| T2 | Declining | 85, 75, 60 | Average: 73.33333333333333; Declined; Declined | Average: 73.33333333333333; Declined; Declined | Pass |
| T3 | No change | 70, 70, 70 | Average: 70.0; No change; No change | Average: 70.0; No change; No change | Pass |
| T4 | Mixed | 100, 89, 99 | Average: 96.0; Declined; Improved | Average: 96.0; Declined; Improved | Pass |

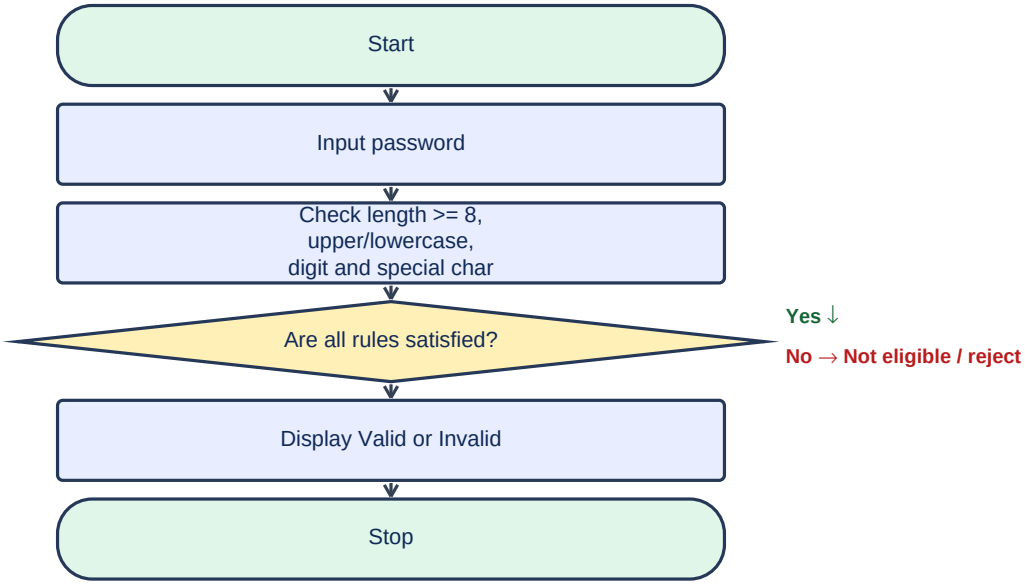
# 9. Password Policy Validation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Check whether a password meets basic length and character requirements.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
p = input("Password: ")
ok = len(p) >= 8 and any(c.isupper() for c in p) and any(c.islower() for c in p)
ok = ok and any(c.isdigit() for c in p) and any(not c.isalnum() for c in p)
print("Valid" if ok else "Invalid")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Password: Hello123@
Output: Valid

## 5. CONCLUSION

Checks password length and character types.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Password: tarangini@452
Invalid
>>>
```

# 9. Password Policy Validation

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To validate text input against several rules.
- To use string methods (isupper, islower, isdigit, isalnum) with any().
- To combine many conditions into a single Valid or Invalid decision.

## 8. ALGORITHM

1. START.
2. Read the password.
3. Check that the length is at least 8 characters.
4. Check that it has at least one uppercase letter and one lowercase letter.
5. Check that it has at least one digit and one special character.
6. IF all rules are satisfied THEN display "Valid".
7. ELSE display "Invalid".
8. STOP.

## 9. PSEUDOCODE

```
BEGIN
  INPUT p
  ok <- length(p) >= 8
  ok <- ok AND p has an uppercase letter AND p has a lowercase letter
  ok <- ok AND p has a digit AND p has a special character
  IF ok THEN PRINT "Valid"
  ELSE PRINT "Invalid"
END
```

## 10. ASSUMPTIONS

- The password is entered as plain text.
- The policy needs at least 8 characters, an uppercase letter, a lowercase letter, a digit and a special character.
- A special character means any character that is not a letter or a digit.
- No other checks (common words, repeated characters) are performed.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Valid | Hello123@ | Valid | Valid | Pass |
| T2 | No uppercase | tarangini@452 | Invalid | Invalid | Pass |
| T3 | No special char | Hello1234 | Invalid | Invalid | Pass |
| T4 | Too short | Hi1@ | Invalid | Invalid | Pass |
| T5 | No digit | Hello@@@@ | Invalid | Invalid | Pass |

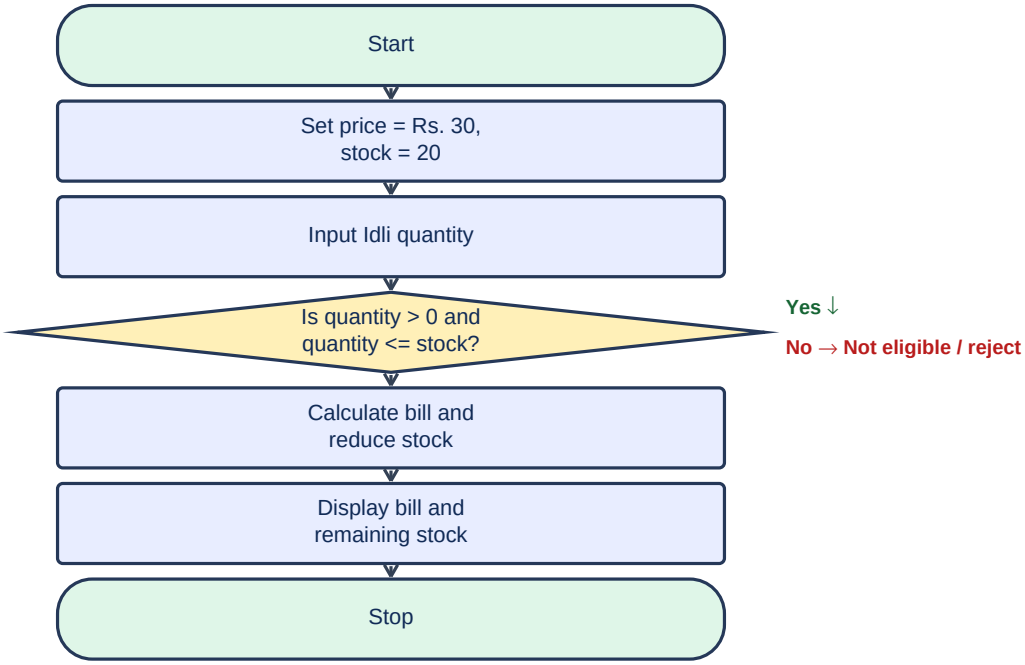
# 10. Smart Canteen Order and Stock

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 1. PROBLEM STATEMENT

Calculate an order bill and check whether enough food items remain in stock.

## 2. FLOWCHART




## 3. PYTHON SOURCE CODE

```
price = 30
stock = 20
qty = int(input("Idli quantity: "))
if 0 < qty <= stock:
print("Bill:", price * qty)
stock = stock - qty
print("Remaining stock:", stock)
else:
print("Invalid quantity or insufficient stock")
```

## 4. SAMPLE INPUT AND OUTPUT

Input: Idli quantity: 2

Output: Bill: 60; Remaining stock: 18

## 5. CONCLUSION

Calculates the order bill and updates the stock.

## 6. OUTPUT SCREENSHOT

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/Tarangini/OneDrive/Desktop/python lab/activity.py =====
Idli quantity: 5
Bill: 150
Remaining stock: 15
>>>
```

# 10. Smart Canteen Order and Stock

PYTHON ACTIVITY-BASED LEARNING • CLEAR FLOWCHART

## 7. OBJECTIVES

- To calculate an order bill from a fixed price and quantity.
- To check available stock before accepting an order.
- To update the stock value after a successful order.

## 8. ALGORITHM

1. START.
2. Set price = 30 and stock = 20.
3. Read the idli quantity.
4. IF quantity > 0 AND quantity <= stock THEN display bill = price x quantity, reduce stock by quantity and display the remaining stock.
5. ELSE display "Invalid quantity or insufficient stock".
6. STOP.

## 9. PSEUDOCODE

```
BEGIN
  price <- 30
  stock <- 20
  INPUT qty
  IF qty > 0 AND qty <= stock THEN
    PRINT price * qty
    stock <- stock - qty
    PRINT stock
  ELSE
    PRINT "Invalid quantity or insufficient stock"
  ENDIF
END
```

## 10. ASSUMPTIONS

- The price of one idli is fixed at Rs. 30 and the starting stock is 20.
- The quantity is a whole number entered by the customer; one order is processed per run.
- An order is accepted only if the quantity is greater than zero and not more than the stock.
- Stock is not carried over between runs because it is reset to 20 each time.

## 11. TEST CASES

| TC | Scenario | Input | Expected output | Actual | Status |
|---|---|---|---|---|---|
| T1 | Normal order | 2 | Bill: 60; Remaining stock: 18 | Bill: 60; Remaining stock: 18 | Pass |
| T2 | Order of 5 | 5 | Bill: 150; Remaining stock: 15 | Bill: 150; Remaining stock: 15 | Pass |
| T3 | Full stock | 20 | Bill: 600; Remaining stock: 0 | Bill: 600; Remaining stock: 0 | Pass |
| T4 | More than stock | 21 | Invalid quantity or insufficient stock | Invalid quantity or insufficient stock | Pass |
| T5 | Zero quantity | 0 | Invalid quantity or insufficient stock | Invalid quantity or insufficient stock | Pass |